React.js Hooks

Hooks

- **In React, Hooks** are **reusable functions** that **provide access to state** in **React Applications.**
- **Hooks** were introduced in the **16.8 version of React**.
- Hooks give access to states for functional components while creating a React application.
- It allows you to use state and other React features without writing a class.
- React Hooks provide **functional components** with the ability to **use states and manage side effects.**
- However, you can now use a Hook within the existing function component to add a state.

Hooks Types

1. **State Hooks**
2. **Context Hooks**
3. **Ref Hooks**
4. **Effect Hooks**
5. **Performance Hooks**
6. **Resource Hooks**

1. State Hooks

State Hooks stores and provide access to the information. To add state in Components we use:

- **useState Hook:** useState Hooks provides state variable with direct update access.
- **useReducer Hook:** useReducer Hook provides a state variable along with the update logic in reducer function.

useState

- **useState** is a basic React hook, which allows a function component **to maintain its own state** and **re-render itself based** on the state changes.
- The signature of the **useState** is as follows −

const [<state>, <setState>] = useState(<initialValue>)

- **initialValue** − Initial value of the state. state can be specified in any type (number, string, array and object).

- **state** − Variable to represent the value of the state.

- **setState** − Function variable to represent the function to update the state returned by the useState.

setState

- The signature of the **setState** function is as follows −

setState(<valueToBeUpdated>)

- **valueToBeUpdated** is the value to be updated for the state.

useState and setState

- The sample usage **to set and update the user's name** is as follows −

```
// initialize the state
const [name, setName] = useState('Rohan')

// update the state
setName('Rohit')
```

Using useState Hook - In App.js

```
import React, { useState } from 'react';

function App() {
    const click = useState('GeeksForGeeks');
    return (
        <h1>Welcome to {click}</h1>
    );
}

export default App;
```

Updating React useState Hook State- In App.js

```
import React, { useState } from 'react';

function App() {
    const [click, setClick] = useState(0);
     // to assign initial value 0
    // to click and a reference to the function that updates click to setClick
    return (
        <div>
            <p>You clicked {click} times</p>

            <button onClick={() => setClick(click + 1)}>
                Click me
            </button>
        </div>
    ); } export default App;
```

=> for instantly updating the contents

Updating React useState Hook State- In App.js

```
import React, { useState } from 'react';

function App() {
    const [click, setClick] = useState(0);
     // to assign initial value 0
    // to click and a reference to the function that updates click to setClick
    return (
        <div>
            <p>You clicked {click} times</p>

            <button onClick={() => setClick(click + 1)}>
                Click me
            </button>
        </div>
    ); } export default App;
```

=> for instantly updating the contents

MUI Components - Dialog

add the code in App.js

```
import React, { useState } from 'react';
import Dialog from '@mui/material/Dialog';
import DialogActions from '@mui/material/DialogActions';
import DialogContent from '@mui/material/DialogContent';
import DialogContentText from '@mui/material/DialogContentText';
import DialogTitle from '@mui/material/DialogTitle';
import Button from '@mui/material/Button';

function MyDialog() {
  const [open, setOpen] = useState(false);

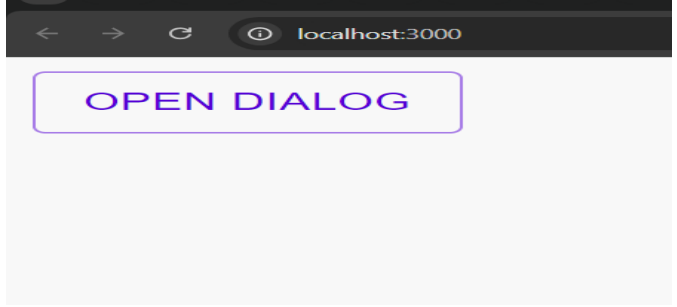
  const handleClickOpen = () => {
    setOpen(true);
  };

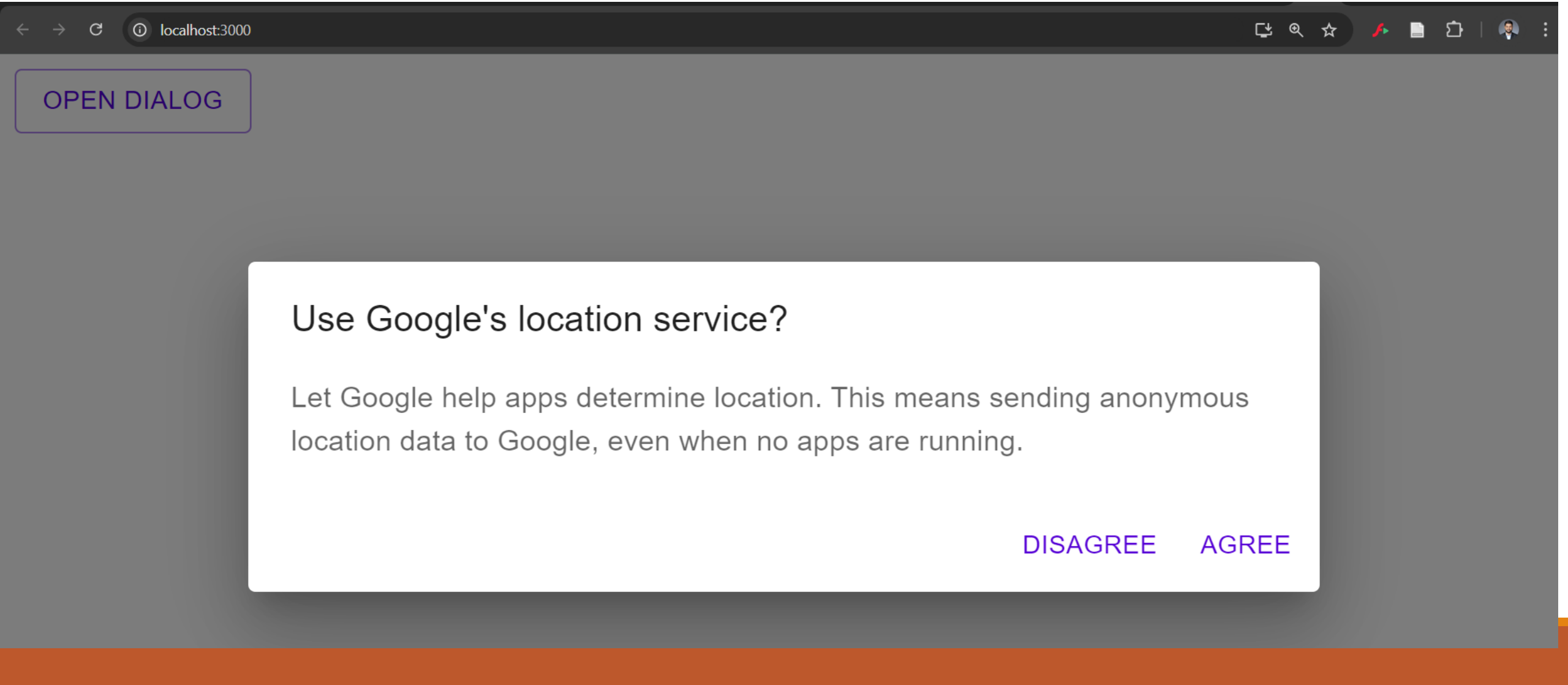
  const handleClose = () => {
    setOpen(false);
  };
```

MUI Components - Dialog

add the code in App.js

```
return (
  <div>
    <Button variant="outlined" onClick={handleClickOpen}>
      Open Dialog
    </Button>
    <Dialog open={open} onClose={handleClose}>
      <DialogTitle>{"Use Google's location service?"}</DialogTitle>
      <DialogContent>
        <DialogContentText>
          Let Google help apps determine location.
        </DialogContentText>
      </DialogContent>
      <DialogActions>
        <Button onClick={handleClose} color="primary">
          Disagree
        </Button>
        <Button onClick={handleClose} color="primary" autoFocus>
          Agree
        </Button>
      </DialogActions>
    </Dialog>
  </div>
 );
}
export default MyDialog;
```


localhost:3000
OPEN DIALOG


localhost:3000
OPEN DIALOG
Use Google's location service?
Let Google help apps determine location. This means sending anonymous location data to Google, even when no apps are running.
DISAGREE
AGREE

MUI Components - Snackbar

add the code in App.js

```
import React, { useState } from 'react';
import Button from '@mui/material/Button';
import Snackbar from '@mui/material/Snackbar';

function MySnackbar() {
  const [open, setOpen] = useState(false);

  const handleClick = () => {
    setOpen(true);
  };

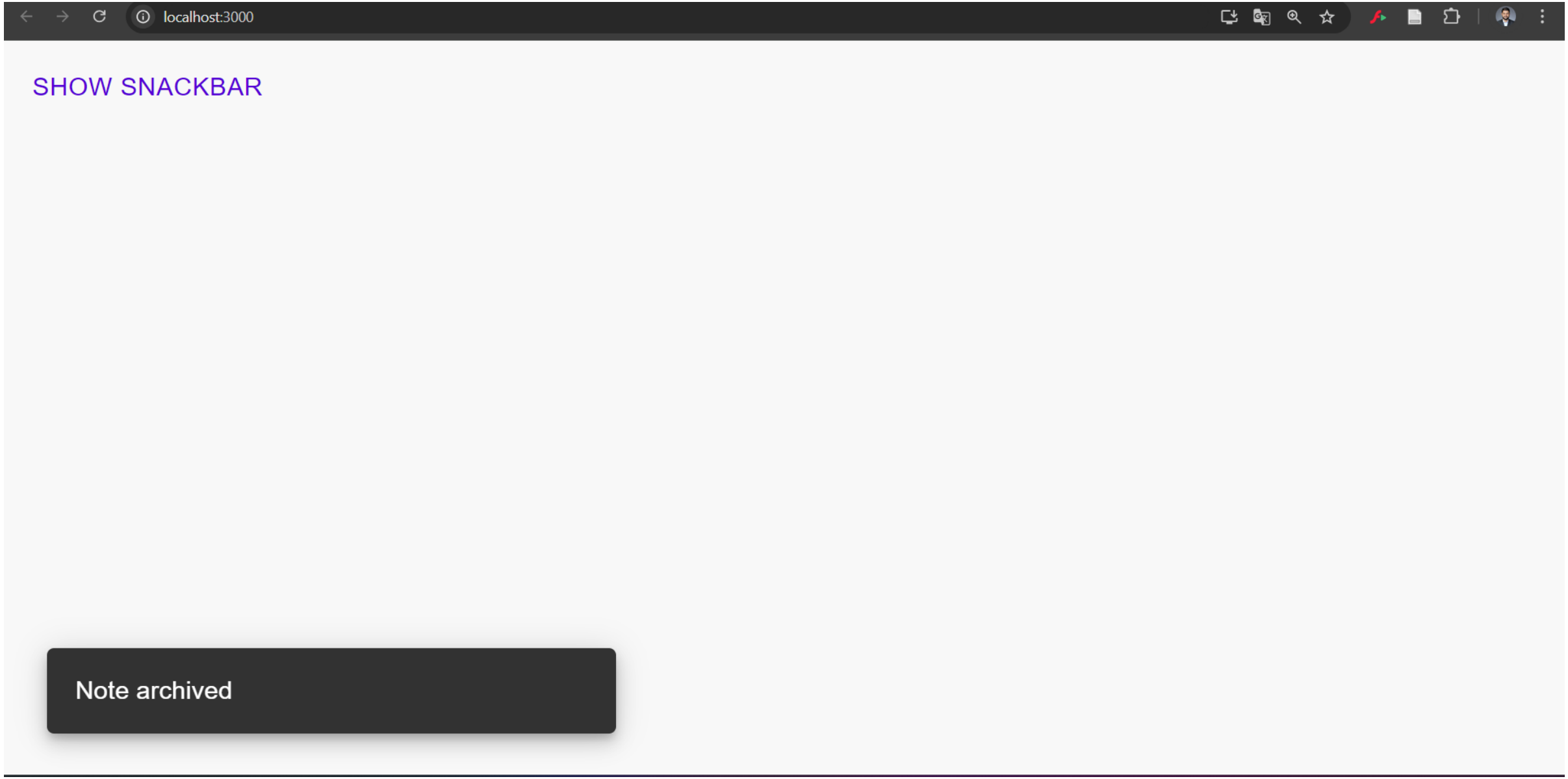
  const handleClose = (event, reason) => {
    if (reason === 'clickaway') {
      return;
    }
    setOpen(false);
  };
```

MUI Components - Snackbar

add the code in App.js

```
return (
  <div>
    <Button onClick={handleClick}>Show Snackbar</Button>
    <Snackbar
      open={open}
      autoHideDuration={6000}
      onClose={handleClose}
      message="Note archived"
    />
  </div>
 );
}

export default MySnackbar;
```


localhost:3000
SHOW SNACKBAR
Note archived

2. Context Hooks

Context hooks make it possible to access the information without being passed as a prop.

3. Ref Hooks

Refs creates the variable containing the information not used for rendering

4. Effect Hooks

Effects connect the components and make it sync with the system.

5. Performance Hooks

Performance hooks are a way to skip the unnecessary work and optimize the rendering performance.

6. Resource Hooks

It allows component to access the resource without being a part of their state